

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Branch: CSM

Section: B

Task No.: 3

Student Name: M. Niharika

Roll No.: A24126552097

1. TITLE

Build an Interactive To-Do List Application Using JavaScript DOM Manipulation

2. AIM

To build an interactive to-do list web application that lets a user add, mark complete, and delete tasks dynamically using **JavaScript DOM manipulation**, without reloading the page.

3. OBJECTIVE

The objectives of this experiment are:

- To capture user input from a text field using the .value property.
 - To validate input and alert the user when the task field is left empty.
 - To dynamically create list elements using document.createElement().
 - To append dynamically created elements to the page using appendChild().
 - To handle click events on dynamically created buttons using addEventListener().
 - To toggle a CSS class with classList.toggle() to visually mark a task as completed.
 - To remove an element from the DOM using removeChild().
 - To conditionally show or hide an empty-state message based on the task count.
 - To apply CSS3 Flexbox styling for a clean, card-based task list layout.
-

4. THEORY

This program is event-driven: nothing happens until the user clicks a button or presses a key. Each task the user adds is represented as a list item (**) built entirely with JavaScript at runtime, rather than written directly in the HTML.

Every list item is created with its own **Complete** and **Delete** buttons, and each button's click listener is defined inside the same function call that created the item. This means the listener automatically "remembers" which specific `<li>` it belongs to — a JavaScript concept known as a closure — so clicking Delete on one task never affects another.

The key methods and concepts used in this program are:

- `document.createElement("li"/"span"/"button"/"div")` creates a new, empty HTML element in memory for each part of a task row.
- `appendChild()` attaches a created element as a child of another element, building the row and then attaching it to the visible list.
- `addEventListener("click", fn)` runs a function whenever a button is clicked; used for the Add, Complete, and Delete buttons.
- `classList.toggle("completed")` adds the completed class if it is absent and removes it if present, switching the strikethrough style on and off.
- `removeChild()` detaches a specific list item from its parent `<ul>`, deleting the task from the page.
- `.trim()` strips leading and trailing whitespace so a task made only of spaces is treated as empty.
- `e.key === "Enter"` lets the Enter key trigger `addTask()`, the same as clicking the Add Task button.
- `style.display` toggles the visibility of the "No tasks available." message based on how many `<li>` elements remain.

The basic flow of the page is:

User types a task and clicks Add Task (or presses Enter) → `addTask()` runs → the input is validated → an `<li>` with task text, a Complete button, and a Delete button is built with DOM methods → the `<li>` is appended to the list → the input is cleared → the empty-state message is shown or hidden → Complete toggles a class on that `<li>`; Delete removes it from the DOM.

5. ALGORITHM / PROCEDURE

- Create two files: `index.html` and `script.js`, and link `script.js` at the end of the body.
- Add a text input for the task name, an "Add Task" button, an empty `<ul>` for the task list, and an empty-state `<p>` message.
- In `script.js`, select the input, button, list, and empty-message elements using `document.getElementById()`.
- Define an `updateEmptyMessage()` function that shows the empty-state message only when the list has no `<li>` children.
- Define an `addTask()` function that reads and trims the input value, and alerts the user if it is blank.
- Inside `addTask()`, create an `<li>`, a `<span>` for the task text, and a Complete and a Delete button, and set the text/labels.

- Attach a click listener to the Complete button that toggles a "completed" class on its own .
- Attach a click listener to the Delete button that removes its own from the list and calls updateEmptyMessage().
- Append the buttons to a container, append the span and container to the , and append the to the list.
- Clear the input field and call updateEmptyMessage() after a task is added.
- Attach addTask() to the Add Task button's click event and to the Enter key on the input field.
- Call updateEmptyMessage() once on page load so the empty message is correct before any task is added.
- Save the files and open index.html in a browser to test adding, completing, and deleting tasks.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>My To-Do List</title>
<style>
  body { font-family: Arial, sans-serif; text-align: center; padding: 40px; background:
#f2f4f8; }
  .input-row { display: flex; justify-content: center; gap: 10px; margin-bottom: 25px; }
  #taskInput { padding: 10px 14px; font-size: 15px; border: 1px solid #ccc; border-radius:
6px; width: 260px; }
  #addTaskBtn { background: #2f6fed; color: #fff; border: none; padding: 10px 20px; border-
radius: 6px; cursor: pointer; }
  #taskList { list-style: none; padding: 0; width: 380px; margin: 0 auto; }
  #taskList li { display: flex; align-items: center; justify-content: space-between;
background: #fff; border-radius: 8px; box-shadow: 0 2px 8px
rgba(0,0,0,0.08);
padding: 10px 15px; margin-bottom: 10px; }
  #taskList li.completed span.taskText { text-decoration: line-through; color: #999; }
  .completeBtn { background: #2fb380; color: #fff; border: none; padding: 6px 12px; border-
radius: 5px; cursor: pointer; }
  .deleteBtn { background: #e5533d; color: #fff; border: none; padding: 6px 12px; border-
radius: 5px; cursor: pointer; margin-left: 6px; }
  #emptyMessage { color: #777; font-style: italic; }
</style>
</head>
<body>

<h1>My To-Do List</h1>
<div class="input-row">
  <input type="text" id="taskInput" placeholder="Enter a task...">
  <button id="addTaskBtn">Add Task</button>
</div>
```

```

<ul id="taskList"></ul>
<p id="emptyMessage">No tasks available.</p>

<script src="script.js"></script>
</body>
</html>

```

script.js

```

const taskInput = document.getElementById("taskInput");
const addTaskBtn = document.getElementById("addTaskBtn");
const taskList = document.getElementById("taskList");
const emptyMessage = document.getElementById("emptyMessage");

function updateEmptyMessage() {
  emptyMessage.style.display = taskList.children.length === 0 ? "block" : "none";
}

function addTask() {
  const taskText = taskInput.value.trim();
  if (taskText === "") { alert("Please enter a task before adding."); return; }

  const li = document.createElement("li");
  const span = document.createElement("span");
  span.className = "taskText";
  span.textContent = taskText;

  const btnGroup = document.createElement("div");

  const completeBtn = document.createElement("button");
  completeBtn.className = "completeBtn";
  completeBtn.textContent = "Complete";
  completeBtn.addEventListener("click", () => li.classList.toggle("completed"));

  const deleteBtn = document.createElement("button");
  deleteBtn.className = "deleteBtn";
  deleteBtn.textContent = "Delete";
  deleteBtn.addEventListener("click", () => {
    taskList.removeChild(li);
    updateEmptyMessage();
  });

  btnGroup.appendChild(completeBtn);
  btnGroup.appendChild(deleteBtn);
  li.appendChild(span);
  li.appendChild(btnGroup);
  taskList.appendChild(li);

  taskInput.value = "";
  updateEmptyMessage();
}

addTaskBtn.addEventListener("click", addTask);
taskInput.addEventListener("keydown", (e) => { if (e.key === "Enter") addTask(); });

updateEmptyMessage();

```

7. CODE EXPLANATION

Code	Explanation
<code>document.getElementById()</code>	Selects a specific element from the page by its id attribute.
<code>addEventListener("click", fn)</code>	Runs the given function whenever the Add, Complete, or Delete button is clicked.
<code>.value / .trim()</code>	Reads the current text typed into the input field and strips extra whitespace.
<code>document.createElement()</code>	Creates a new, empty HTML element (li, span, div, button) in memory.
<code>element.textContent</code>	Sets the visible text inside an element, such as the task name or button label.
<code>appendChild()</code>	Attaches a created element as a child of another element in the DOM.
<code>classList.toggle("completed")</code>	Adds or removes the completed class on a task, switching the strikethrough style.
<code>removeChild()</code>	Detaches a specific list item from the , deleting that task from the page.
<code>e.key === "Enter"</code>	Checks whether the key pressed in the input field was Enter, to add a task without clicking.
<code>style.display</code>	Shows or hides the "No tasks available." message depending on the task count.

8. TEST CASES AND EXECUTION DATA

The page was opened in a browser and the app was tested with different inputs and actions; the results were recorded below.

Test Case	Action	Expected Result	Actual Result	Status
TC-01	Open index.html in a browser	Empty task list is shown with the "No tasks available." message	Empty state displayed correctly	Pass
TC-02	Click "Add Task" with the input left empty	An alert asks the user to enter a task before adding	Alert shown as expected	Pass
TC-03	Type a task and click "Add Task"	The task appears as a new row with Complete and Delete buttons; the empty message disappears	Task added correctly	Pass
TC-04	Type a task and press Enter instead of clicking	The task is added the same way as clicking "Add Task"	Task added via Enter key	Pass

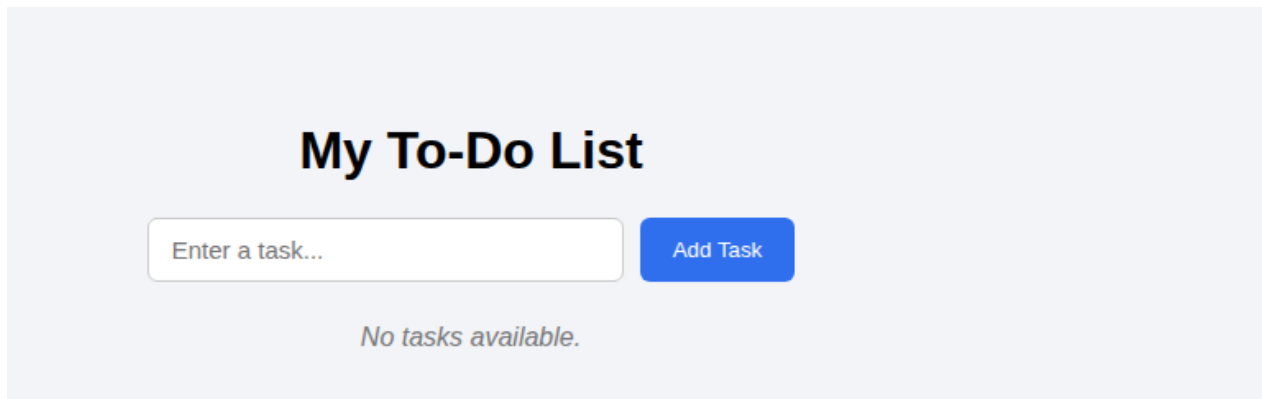
Test Case	Action	Expected Result	Actual Result	Status
TC-05	Click "Complete" on a task	The task text gets a strikethrough and turns grey	Strikethrough style applied	Pass
TC-06	Click "Complete" again on the same task	The strikethrough is removed and the text returns to normal	Style toggled off correctly	Pass

Additional Validation Test

Test Case	Input / Action	Expected Result	Status
TC-07	Click "Delete" on a task	The task row is removed from the list immediately	Pass
TC-08	Delete every task from the list	The "No tasks available." message reappears once the list is empty	Pass

9. OUTPUT

Output 1: Empty To-Do List



My To-Do List

Enter a task... Add Task

No tasks available.

Output 2: Tasks Added, One Marked Complete

My To-Do List

Add Task

~~Buy groceries~~

CompleteDelete

Complete JS assignment

CompleteDelete

Read a book

CompleteDelete

10. RESULT

Thus, an interactive to-do list application was successfully built using JavaScript DOM manipulation. Adding, completing, and deleting tasks all worked correctly without reloading the page, and the empty-state message updated correctly as tasks were added and removed. One minor cosmetic issue was observed: after completing several tasks, the Complete button remains labelled "Complete" even for a completed task; changing its label to "Undo" once a task is marked complete would make the toggle behaviour clearer to the user.